

One product, not two — capability profiles instead of an SSA-dinghy fork

Decision, 19 Sep 2026. SSA is *not* split into SSA-dinghy and SSA-yacht. One codebase, one platform, one data model. What varies between an ILCA and the Northstar 76 is handled by a **capability profile** on the boat, and by two marketing front doors over the same app.

This note records the reasoning so it does not get re-litigated every time the dinghy work touches something built for the N76.

Companion: [dinghy-gps-prior-art-and-twd-2026-09.md](#), which is where the dinghy/GPS analysis and the TWD method ladder live.

1. The decision

Repo	one. No <code>ssa-dinghy</code> .
Data model	one. Same <code>LogRow</code> , same sessions, same campaign spine.
Analysis pipeline	one. The dinghy path <i>synthesises</i> the missing channels and rejoins it.
UI surface	gated per boat by declared capabilities.
Marketing	two front doors, two plans, one product.

2. Why not split

The contrast that settles it is already in this account: `ssa-finance`. Splitting that out was right *because it shares almost nothing but infra* — different data, different users, different domain. Dinghy vs yacht is the exact opposite case. The two would share:

- the weather stack (forecast deck, 3D wind field, MOS, windweight, venue model)
- the media pipeline — photos, video, drone, Bunny, the day's whole record
- the campaign spine, sessions, events, seasons
- auth, teams, invitations, sharing
- tagging, the debrief, the AI analyst

That is the large majority of the code and **essentially all of the differentiation** (see §4 of the prior-art doc: the weather depth, the media record and campaign continuity are what nobody else combines). A fork duplicates everything valuable in order to separate the one layer that differs.

And that layer is smaller than it looks. The whole design thesis of the dinghy work is that the GPS-only case *converges* onto the existing pipeline: manufacture `twd / tws / twa` and a phase list, write them into the existing open `LogRow`, and `phaseStats`, `manoeuvres` and `startAnalysis` run unchanged. If the goal is explicitly "do not fork the analysis", forking the repo contradicts it.

Two of everything is the real cost. Two deploys, two migration histories, two auth setups, two billing integrations, two dependency upgrade treadmills — carried by a team of one. Every shared fix becomes a cherry-pick.

3. Boat type is the wrong axis

What actually varies is **which channels exist**:

boat	instruments	wind sensor	polar	lidar / sail shape	fleet data
Northstar 76	full Expedition	yes	measured	yes	TracTrac at events
TP52	full	yes	measured	sometimes	yes
ORC cruiser-racer	partial	yes	published (ORC)	no	no
Melges 24 + Vakaros	GPS + heading + heel	no	learned	no	sometimes
49er (training)	GPS only	no	learned	photos only	squad boats
ILCA (racing)	none legal	no	learned	photos only	event tracker

This is a **spectrum of available channels, not two buckets**. Split by boat type and the Melges-with-a-Vakaros needs a third product; add a coach-boat anemometer to a dinghy squad and it needs a fourth. Capability is the axis that actually predicts what the app can show.

4. The mechanism: a capability profile

Declare what a boat *has*; let every view state what it *needs*.

```
// boat.specs.capabilities – JSONB, no migration, same pattern as log_profile
interface BoatCapabilities {
  channels: LogField[]           // what the logs actually carry
  wind: 'measured' | 'derived' | 'model' // how twd/tws get there
  polar: 'measured' | 'published' | 'learned' | 'none'
  sailShape: 'lidar' | 'photo' | 'none'
  fleet: 'tracker' | 'squad' | 'none'
  classRules?: { electronicsWhileRacing: boolean } // ILCA et al – see §3 of the prior-art doc
}
```

Rules:

1. **A view declares its required channels** and hides, or degrades to a documented fallback, when they are absent. A dinghy sailor never sees bobstay load; an N76 trimmer never sees an inferred-wind confidence band.
2. **Provenance travels with every derived channel** — measured / derived / modelled / unavailable. This is *not new machinery*: §13d of the prior-art doc requires exactly this for the wind sources, and §8 requires the "unmeasured" state rather than an invented number. Built once, used twice.
3. **The profile is data, not code**. Adding a class is a row, not a release.
4. **It lives next to `log_profile` in `boat.specs`** — same JSONB, no migration. `logProfile.ts` and `hasOpenableData.js` already reach for this shape; this makes it explicit.

5. Marketing is a separate question, and there the answer *is* two

Architecture says one product. Positioning says address two buyers:

- **SSA for Olympic teams** — squad training days, GPS trackers, venue knowledge, media sharing, derived wind. Price point in the €150–600/yr-per-coach band the competitors occupy.
- **SSA for grand-prix yachts** — full instruments, lidar sail shape, rig loads, KND-style phase reporting.

Different landing pages, vocabulary, screenshots and plans over **the same app**. The precedent is in the prior-art survey: Njord sells to TP52, Maxi72, RC44 *and* Olympic dinghies on one platform, and kTool spans dinghies, foilers, skiffs and keelboats. The credible Olympic tools are the ones that do both — nobody in that survey has won by splitting.

6. The one real architectural difference

The dinghy path wants to be **fleet-scoped**: a session is a day at a venue with N boats, and the wind estimator's accuracy depends on pooling them (3–5° single boat → ~1° with five, §5-L5 of the prior-art doc). The yacht path today is boat-first.

This is a genuine difference — but it is **not a reason to split**, because the yacht side wants fleet analysis too: that is what the TracTrac integration is for (gains/losses vs the fleet). It is shared infrastructure that the dinghy case needs *first* and more acutely, not a second product.

Consequence: build the multi-boat session model once, in the shared core.

7. When to revisit

Not "never". Revisit if any of these becomes true:

- shared code genuinely falls below roughly half
- investment or a sale is taken against one line specifically
- a separate team owns one of them
- the session data model has to diverge at the root, not at the edges

None is true today.

8. What to do first

Build the capability profile before the dinghy ingestion work, not after. Retrofitting provenance and channel-gating onto views that assume the N76's channels is the expensive version, and the wind work needs the same machinery anyway. Order:

1. `BoatCapabilities` on `boat.specs`, populated for the N76 (everything) and one dinghy (GPS only).
2. Channel-requirement declarations on the existing tabs/charts, so an under-equipped boat degrades visibly instead of rendering empty axes.
3. Then the tracker ingestion and the wind synthesis layer.